Kubernetes

Support de révision personnel

Du Pod aux Services : cours, commandes, TP et questions d'entretien

---

Hicham Habchane

Master 2 — Préparation à un entretien de stage DevOps

Cluster d'apprentissage : minikube v1.39.0 (driver docker), Kubernetes v1.37.0, kubectl v1.37.1
Ubuntu 26.04, Docker 29.9

Contenu arrêté au 20 octobre 2026 (Programme DevOps, Jour 1, avant le défi final)

Table des matières

Comment lire ce document **2**

1 Kubernetes et son architecture **3**

2 Minikube et kubectl **4**

3 Les Pods **5**

4 Les Deployments **7**

5 Les ReplicaSets et les replicas **9**

6 Le scaling **12**

7 Rolling updates et rollbacks **13**

8 Les Services : ClusterIP, NodePort et port-forward **15**

9 Labels, selectors, Endpoints et EndpointSlices **18**

10 Autres notions étudiées **19**

10.1 Déclaratif, impératif et `apply` 19
10.2 Namespaces et DNS 19
10.3 Pod client jetable 19
10.4 Outils du terminal utilisés 19
10.5 Git et publication sécurisée 19
10.6 Erreurs de méthode rencontrées 20

11 Questions Kubernetes pour un entretien DevOps M2 **20**

11.1 Pods 20
11.2 Deployments 20
11.3 ReplicaSets 21
11.4 Services 21
11.5 Réseau et dépannage 21

12 Fiche récapitulative des commandes **22**

Comment lire ce document

Sources utilisées. Les README de chaque dossier (`01-bases` à `06-services`), les fichiers YAML du projet, mes notes personnelles, `commandes.md`, `PROGRESSION_DEVOPS.md` et l'historique de nos sessions.

Repères.

— [TP] : réellement fait ou réellement posé pendant les TP.
— [COMPLÉMENT] : notion, question ou commande ajoutée pour l'entretien, *pas encore pratiquée*.

Ce qui n'est pas encore fait.

— Le défi final du Jour 1, *sans aide* : Deployment `boutique` (`httpd:2.4`, 3 replicas), Service NodePort sur 30090, rolling update vers `httpd:2.4-alpine`, rollback, fichier remis d'accord avec le cluster, site testé. Le Jour 1 n'est donc *pas encore validé*.
— ConfigMaps, Secrets, probes, ressources, volumes, Ingress (Jour 2 et suivants).

Informations partielles. Les sorties complètes des toutes premières séances (installation) ne sont plus toutes disponibles. Seuls les résultats notés dans les README sont repris ici. Lorsqu'une commande a été discutée sans que son exécution soit attestée, c'est indiqué.

Le fil conducteur : le restaurant.

Kubernetes	**Analogie**
Conteneur	le plat
Pod	le plateau qui porte le plat
ReplicaSet	le chef d'équipe qui compte les plateaux
Deployment	le responsable de salle (il change aussi la vaisselle en plein service)
template	la fiche recette
labels	l'autocollant collé sur chaque plateau
Service	l'accueil du restaurant
Nœud / cluster	une cuisine / le restaurant entier
Kubernetes / kubectl	le gérant / la télécommande

1. Kubernetes et son architecture

A. Résumé du cours

Docker met une application dans un conteneur et le lance. Si ce conteneur plante, personne ne le relance. **Kubernetes** orchestre les conteneurs : il les lance, les multiplie, les surveille, les relance et les répartit sur des machines. Les deux ne sont pas concurrents : Kubernetes fait tourner des conteneurs.

L'état souhaité est l'idée centrale. Je ne donne pas un ordre (« lance un conteneur ») : je *décris* ce que je veux (« toujours 3 nginx »). Kubernetes compare en permanence la réalité à cette description et corrige les écarts : c'est la *boucle de réconciliation*.

	Docker	**Kubernetes**
Rôle	crée et lance des conteneurs	organise et surveille beaucoup de conteneurs
Conteneur qui plante	reste arrêté	recréé automatiquement
Plusieurs copies	à la main	« je veux 5 copies »
Échelle	une machine	un cluster (une ou plusieurs machines)

Les composants réellement observés.

— **API server** : la porte d'entrée du cluster. `kubectl` lui parle en HTTPS. Dans le cluster, il est joignable via le Service `kubernetes` (`10.96.0.1:443`).
— **default-scheduler** : choisit le nœud où placer chaque Pod (vu dans `describe pod`).
— **kubelet** : sur chaque nœud, télécharge l'image et démarre les conteneurs.
— **containerd** : le moteur de conteneurs utilisé par minikube (et non Docker).
— **Contrôleurs** (`replicaset-controller`, `deployment-controller`) : ils appliquent la boucle de réconciliation (vus dans les `Events`).
— **CoreDNS** : le DNS du cluster, qui traduit un nom de Service en IP. C'est un Pod du namespace `kube-system`.

[COMPLÉMENT] Les autres composants du control-plane sont **etcd** (la base qui stocke l'état du cluster) et le **controller-manager** (il regroupe les contrôleurs). **kube-proxy** programme, sur chaque nœud, la redirection du trafic des Services.

B. Commandes importantes

```
docker run -d --name monsite nginx   # lance un conteneur nginx en arrière-plan
docker ps                            # liste les conteneurs en marche
docker kill monsite                  # arrête brutalement le conteneur (panne simulée)
docker rm monsite                    # supprime le conteneur arrêté
```

C. Exercices pratiques

Exercice : le problème de Docker seul [tp]

Objectif. Montrer que Docker ne relance pas un conteneur tombé.
Commandes. `docker run`, `docker ps`, `docker kill`, `docker ps`, `docker rm`.
Résultat. Après `docker kill`, `monsite` disparaît de `docker ps` et personne ne le relance.
Appris. Il faut un « gérant » qui surveille et corrige : c'est le rôle de Kubernetes.

D. Questions et réponses [tp]

Q. Quelle est la différence entre Docker et Kubernetes ?

R. Docker crée et lance les conteneurs ; Kubernetes les organise : il les lance, les multiplie, les surveille, les relance automatiquement et les répartit sur les machines.

Q. Qu'est-ce que l'état souhaité ?

R. Je décris ce que je veux en permanence (« toujours 3 nginx ») et Kubernetes s'arrange pour que la réalité y corresponde.

Q. D'après la colonne `From` de `describe pod`, qui choisit le nœud où placer le Pod ?

R. Le `default-scheduler`. Le kubelet, lui, télécharge l'image et démarre le conteneur.

Q. À quoi sert le Service `kubernetes` (port 443), que je n'ai pas créé ?

R. C'est l'accès à l'API server, en HTTPS. Kubernetes le crée lui-même ; les Pods qui veulent parler à l'API passent par lui. Il ne faut pas le supprimer.

E. Erreurs et dépannage

— **Confusion kubectl / Kubernetes.** J'ai d'abord dit que kubectl « déploie et surveille ». Correction : kubectl n'est que la télécommande ; c'est Kubernetes qui déploie et maintient les répliques.

2. Minikube et kubectl

A. Résumé du cours

minikube crée un petit cluster Kubernetes sur mon PC, à l'intérieur d'un conteneur Docker : c'est un restaurant miniature pour s'entraîner. **kubectl** est la télécommande en ligne de commande : il envoie mes demandes à l'API server. Mon cluster a **un seul nœud**, `minikube`, qui pilote le cluster (control-plane) et fait tourner les applications. Ce n'est pas un cluster de production : pas de haute disponibilité.

B. Commandes importantes

```
kubectl version --client          # version de kubectl (v1.37.1)
minikube version                  # version de minikube (v1.39.0)
minikube start --driver=docker    # crée ou rallume le cluster dans un conteneur Docker
minikube start                    # rallume le cluster après un redémarrage du PC
minikube status                   # host / kubelet / apiserver : Running ou Stopped
minikube stop                     # éteint le cluster (libère la RAM, garde les objets)
minikube ip                       # IP du œnud (192.168.49.2), utile pour NodePort
kubectl get nodes                 # liste les œnuds ; attendre STATUS = Ready
```

C. Exercices pratiques

Installation sans sudo [tp]

Objectif. Installer les outils et démarrer un cluster.
Résultat. kubectl v1.37.1 et minikube v1.39.0 installés dans `~/.local/bin`, vérifiés par somme de contrôle sha256. `minikube start --driver=docker` lance un cluster Kubernetes v1.37.0.
Observé. Le nœud est `NotReady` quelques secondes au démarrage, puis `Ready`.
Appris. Après chaque redémarrage du PC, le cluster est éteint : toujours commencer par `minikube start` puis `kubectl get nodes`.

D. Questions et réponses [tp]

Q. À quoi sert kubectl ?

R. C'est la télécommande en ligne de commande qui envoie mes demandes au cluster (à l'API server). Ce n'est pas lui qui fait le travail.

Q. Combien de nœuds a mon cluster et quel est leur rôle ?

R. Un seul, `minikube` : il pilote le cluster (control-plane) et exécute les applications.

E. Erreurs et dépannage

Symptôme	Cause	Correction
`no route to host` ou `connection refused`	cluster éteint après le redémarrage du PC	`minikube status` puis `minikube start`
Nœud `NotReady`	le nœud démarre encore	attendre, relancer `kubectl get nodes`

3. Les Pods

A. Résumé du cours

Un **Pod** est la plus petite unité gérée par Kubernetes : une enveloppe autour d'un conteneur (parfois plusieurs, qui partagent le même réseau et se parlent par `localhost`). Il a un nom et une IP *interne* au cluster. Un Pod est **jetable** : s'il meurt, on ne le répare pas, on en crée un nouveau (nouveau nom, nouvelle IP). Un Pod créé seul n'est *jamais* relancé.

Un fichier YAML a 4 parties.

`apiVersion`	version de l'API : `v1` (Pod, Service), `apps/v1` (Deployment, ReplicaSet)
`kind`	type d'objet : Pod, Deployment, Service…
`metadata`	nom et étiquettes
`spec`	ce que je veux exactement

En YAML, l'indentation compte : uniquement des espaces, jamais de tabulations.

RESTARTS. Cette colonne compte les redémarrages du *conteneur*, pas du Pod. Même nom et même AGE avec `RESTARTS 1` : le conteneur a redémarré (par exemple après `minikube stop/start`). Un nouveau nom signifie au contraire que le Pod a été remplacé.

B. Commandes importantes

```
kubectl apply -f 02-pods/pod-nginx.yaml   # envoie l'état souhaité (created)
kubectl get pods                          # NAME, READY, STATUS, RESTARTS, AGE
kubectl get pods -o wide                  # ajoute l'IP et le œnud
kubectl describe pod mon-premier-pod      # fiche détaillée, Events en bas
kubectl logs mon-premier-pod              # sortie du conteneur (comme docker logs)
kubectl delete pod mon-premier-pod        # supprime le Pod
```

C. Exercices pratiques

Créer, inspecter puis supprimer un Pod seul [tp]

Fichier `02-pods/pod-nginx.yaml`.

```
apiVersion: v1
kind: Pod
metadata:
  name: mon-premier-pod
  labels:
    app: monsite
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
```

Résultats.

— `apply` affiche `pod/mon-premier-pod created`. À 10 s : `READY 0/1`, `ContainerCreating` (téléchargement de l'image). À 21 s : `1/1 Running`.
— `describe` : IP `10.244.0.3` ; 5 Events `Scheduled`, `Pulling`, `Pulled`, `Created`, `Started`.
— `logs` : ligne `Configuration complete; ready for start up`. Les heures des logs sont en UTC.
— Après `delete`, `get pods` répond `No resources found` : le Pod ne revient pas.

Appris. `created` ne veut pas dire « en marche ». Un Pod seul n'a personne pour le surveiller.

D. Questions et réponses [tp]

Q. Kubernetes gère-t-il des Pods ou des conteneurs ?

R. Des Pods. Il ne gère jamais un conteneur « tout nu ».

Q. Si un Pod meurt, que se passe-t-il ?

R. Il n'est pas réparé. Si un contrôleur le gère, celui-ci en crée un nouveau, avec un nouveau nom et une nouvelle IP.

Q. Quelle est l'IP de ton Pod ?

R. `10.244.0.3`, une adresse interne au cluster.

Q. Combien de lignes dans Events, et quelle est la dernière ?

R. 5 lignes ; la dernière est `Started`.

Q. Quelle commande Docker fait la même chose que `kubectl logs` ?

R. `docker logs`.

Q. Quelle ligne montre que nginx est prêt ?

R. `Configuration complete; ready for start up`.

Q. Après `kubectl delete pod`, le Pod revient-il ? Pourquoi ?

R. Non : il faut un Deployment pour le relancer. Un Pod seul n'a pas de contrôleur.

Q. Les Pods ont gardé leur nom mais `RESTARTS` est passé de 0 à 1 : qu'est-ce qui a redémarré ?

R. Le conteneur, pas le Pod. L'objet Pod est conservé (même nom, même AGE) ; ses conteneurs ont été relancés au redémarrage de minikube.

E. Erreurs et dépannage

— **Croire que `created` = `Running`.** Il faut vérifier avec `kubectl get pods` et attendre `Running`.
— **Heure des logs décalée.** Les logs sont en UTC.

4. Les Deployments

A. Résumé du cours

Un **Deployment** surveille les Pods pour moi : « je veux toujours 3 Pods nginx ». S'il en manque un, il en recrée un. Il ne gère pas les Pods directement. La chaîne est **Deployment** → **ReplicaSet** → **Pods**. Il apporte en plus les mises à jour progressives (rolling update) et le retour arrière (rollback).

— `replicas` : le nombre de Pods voulus.
— `template` : la recette de chaque Pod (conteneurs, image, ports, étiquettes).
— `selector.matchLabels` : l'étiquette qui permet de reconnaître « ses » Pods. Elle doit correspondre aux `labels` du template, sinon le fichier est refusé.

Nom d'un Pod : `monsite-86f69f946d-4vcxz`. On y lit le nom du Deployment, puis le hash du template (qui identifie le ReplicaSet, identique pour tous les Pods d'une même version), puis l'identifiant unique du Pod.

B. Commandes importantes

```
kubectl apply -f 04-deployments/deployment-nginx.yaml   # crée ou met à jour
kubectl get deployments                  # READY 3/3 = 3 Pods prêts sur 3 voulus
kubectl get deployments,replicasets,pods # les trois niveaux en une commande
kubectl describe deployment monsite      # stratégie, Events Scaled up/down
kubectl delete pod <nom-d-un-pod>        # panne simulée : un remplaçant arrive
```

C. Exercices pratiques

Créer le Deployment `monsite` [tp]

Fichier `04-deployments/deployment-nginx.yaml` (état actuel : 5 replicas, image `nginx`).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: monsite
spec:
  replicas: 5            # 3 au départ, passé à 5 à l'exercice 5
  selector:
    matchLabels:
      app: monsite
  template:
    metadata:
      labels:
        app: monsite
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Résultats. `deployment.apps/monsite created` ; `READY 3/3` en 8 s ; 3 Pods `monsite-86f69f946d-xxxxx`, avec la même série du milieu.

Supprimer un Pod géré par le Deployment [tp]

Commande. `kubectl delete pod monsite-86f69f946d-f56w5`.
Résultat. Toujours 3 Pods : `f56w5` a disparu et un nouveau Pod `4vcxz` est apparu (AGE 9 s, contre 4 min 47 s pour les autres). Il porte le même hash, donc le même ReplicaSet.
Appris. Contrairement à `docker kill` et au Pod seul, le Pod est remplacé automatiquement.

D. Questions et réponses [tp]

Q. Que veut dire `replicas: 3` ?

R. Le nombre de Pods identiques que Kubernetes maintient en permanence ; si un Pod plante ou est supprimé, il en recrée un.

Q. À quoi sert la partie `template` ?

R. C'est la recette de chaque Pod : conteneurs, image, ports et étiquettes sont copiés dans chaque nouveau Pod.

Q. Comment le Deployment sait-il quels Pods sont les siens ?

R. Grâce au `selector` (`matchLabels: app: monsite`), qui doit correspondre aux étiquettes du template.

Q. Combien de Pods vois-tu ? Que remarques-tu sur leurs noms ?

R. 3 Pods. Ils commencent tous par `monsite` ; la série du milieu est identique (la version du template), seule la dernière change.

Q. Après la suppression de `f56w5` : combien de Pods ? Y a-t-il un nouveau nom ? Comment le reconnaître ?

R. Toujours 3 ; `f56w5` a disparu, remplacé par un Pod au nouveau suffixe, reconnaissable à son AGE très faible.

5. Les ReplicaSets et les replicas

A. Résumé du cours

Un **ReplicaSet** a une seule mission : garder N Pods identiques en marche. Il compte les Pods qui portent *son étiquette* et ne regarde *que* les étiquettes, jamais « qui a créé le Pod ».

— S'il y a trop de Pods, il en supprime. L'ordre : d'abord les Pods non placés, puis les non prêts, puis ceux prêts depuis le moins longtemps.
— S'il trouve un Pod orphelin qui porte la bonne étiquette, il l'**adopte**.
— **Limite** : si l'image du template change, les Pods existants *ne changent pas*. Il ne vérifie que le nombre.

Règle : un sélecteur = un seul contrôleur. En pratique, on n'écrit jamais de ReplicaSet à la main : on écrit un Deployment.

B. Commandes importantes

```
kubectl get replicasets                    # DESIRED / CURRENT / READY
kubectl describe replicaset <nom>          # Controlled By, Selector, Events
kubectl get pods -l app=web                # filtre par étiquette (-l)
kubectl describe pod intrus                # Controlled By: ReplicaSet/web-rs
kubectl delete replicaset web-rs           # supprime le ReplicaSet ET ses Pods
kubectl get pods -o custom-columns=NOM:.metadata.name,IMAGE:.spec.containers[0].image
```

C. Exercices pratiques

Exercice 1 : le ReplicaSet créé par le Deployment [tp]

Résultats. `monsite-86f69f946d` : DESIRED 3, CURRENT 3, READY 3. Dans `describe` : `Controlled By: Deployment/monsite`, et `Selector: app=monsite,pod-template-hash=86f69f946d`. Le Deployment ajoute l'étiquette `pod-template-hash` pour séparer les versions.

Exercice 2 : écrire un ReplicaSet à la main [tp]

Fichier `05-replicasets/replicaset-web.yaml` (écrit seul ; image passée à `nginx:1.27` à l'exercice 4).

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

Résultat. Pods `web-rs-ktz59` et `web-rs-pjdsm`, sans hash au milieu du nom.

Exercice 3A : le Pod intrus [tp]

Fichier `pod-intrus.yaml` : un Pod `intrus`, étiquette `app: web`, créé alors que `web-rs` a déjà ses 2 Pods.
Résultat. Event `SuccessfulDelete …Deleted pod: intrus` : le ReplicaSet a compté 3 Pods `app=web` et en a supprimé un.

Exercice 3B : prouver l'adoption [tp]

```
kubectl delete replicaset web-rs
kubectl apply -f 05-replicasets/pod-intrus.yaml       # l'intrus d'abord
kubectl apply -f 05-replicasets/replicaset-web.yaml   # puis le ReplicaSet
kubectl get pods -l app=web                           # intrus + 1 seul nouveau Pod
kubectl describe pod intrus                           # Controlled By: ReplicaSet/web-rs
```

Appris. Le ReplicaSet adopte un Pod orphelin qui porte son étiquette.

Exercice 4 : la limite du ReplicaSet [tp]

Changement de l'image en `nginx:1.27` dans le fichier, puis `apply` : `configured`, mais les Pods existants restent en `nginx`. Pour forcer la mise à jour, il faut supprimer les Pods un par un. Pendant ce temps, `nginx` et `nginx:1.27` tournent en même temps.

	ReplicaSet seul	**Deployment**
Maintenir N Pods	oui	oui (via un ReplicaSet)
Mettre à jour l'image des Pods existants	non (à la main)	oui (rolling update)
Historique et retour arrière	non	oui

D. Questions et réponses [tp]

Q. Le nom `monsite-86f69f946d` est composé de quoi ?

R. Du nom du Deployment (`monsite`) et du hash du template du Pod.

Q. Quelle ligne de `describe` prouve que ce ReplicaSet appartient au Deployment `monsite` ?

R. `Controlled By: Deployment/monsite`.

Q. Les Pods de `web-rs` ont-ils un hash au milieu de leur nom ? Pourquoi ?

R. Non. C'est le Deployment qui ajoute `pod-template-hash` ; ici le ReplicaSet a été créé directement.

Q. Quelles lignes as-tu changées par rapport au Deployment ?

R. `kind` (ReplicaSet), le nom (`web-rs`), `replicas` (2) et l'étiquette `app: web` dans le selector *et* le template. `apiVersion` reste `apps/v1`.

Q. Pourquoi ne fallait-il pas utiliser `app: monsite` ?

R. Pour éviter que deux contrôleurs se disputent les mêmes Pods. Un sélecteur = un seul contrôleur.

Q. Que fait le ReplicaSet quand l'intrus (`app: web`) apparaît ?

R. Il en supprime un pour revenir à 2 ; c'est l'intrus, le plus récent, qui est supprimé.

Q. L'intrus n'a pas été créé par `web-rs`, et pourtant il a été supprimé. Qu'est-ce que ça t'apprend ?

R. Le ReplicaSet reconnaît ses Pods uniquement par leurs étiquettes.

Q. Si l'intrus existe *avant* `web-rs` (replicas 2), combien de nouveaux Pods sont créés ?

R. Un seul : le ReplicaSet adopte l'intrus.

Q. Quelle ligne prouve l'adoption ?

R. `Controlled By: ReplicaSet/web-rs` dans `describe pod intrus`.

Q. Après le passage à `nginx:1.27` dans le ReplicaSet, quelle image ont les Pods existants ?

R. Toujours `nginx` : le nouveau template ne sert qu'aux Pods créés ensuite.

Q. Avec un ReplicaSet seul, comment forcer le passage à `nginx:1.27` ?

R. En supprimant les Pods : il les recrée avec la nouvelle recette.

Q. Avec 50 Pods en production, quel problème ?

R. Une coupure du site si on les supprime tous d'un coup. Un par un, c'est long, manuel et sans historique : c'est le travail du Deployment.

Q. Deux versions tournent en même temps : quel problème pour un visiteur ?

R. Il voit une version différente selon le Pod qui lui répond. C'est aussi le cas pendant un rolling update : les versions doivent rester compatibles.

6. Le scaling

A. Résumé du cours

Changer `replicas` change seulement le *nombre* de Pods. Comme `replicas` n'est pas dans le template, le hash ne change pas : même ReplicaSet, seul son DESIRED change, et les Pods existants sont conservés. Deux façons de faire :

— **déclarative** (bonne pratique) : modifier le fichier, puis `apply`. Le fichier, versionné dans Git, fait foi ;
— **impérative** : `kubectl scale`. Le cluster change, mais pas le fichier, ce qui crée une **dérive**.

B. Commandes importantes

```
sed -i 's/replicas: 3/replicas: 5/' 04-deployments/deployment-nginx.yaml
grep replicas 04-deployments/deployment-nginx.yaml     # vérifier la modification
kubectl apply -f 04-deployments/deployment-nginx.yaml  # configured
kubectl scale deployment monsite --replicas=2          # impératif (dérive)
```

C. Exercices pratiques

Exercice 5 : passer de 3 à 5 replicas [tp]

Résultats. `configured` ; 5/5 ; les 3 anciens Pods gardent leur nom (`4vcxz`, `kfrzp`, `kjxxs`) et 2 nouveaux arrivent (`85k7z`, `gdk8x`). Le ReplicaSet reste `86f69f946d`, avec DESIRED 5.

Déclaratif ou impératif (discussion) [tp]

Le cas `kubectl scale --replicas=2` a été étudié par prédiction ; son exécution n'est pas attestée dans mes notes.

D. Questions et réponses [tp]

Q. Passage à 5 replicas : combien de Pods ? Les anciens gardent-ils leur nom ? Quel mot affiche `apply` ?

R. 5 Pods (3 anciens et 2 nouveaux) ; les anciens gardent leur nom, car on change le nombre, pas la recette ; `apply` affiche `configured`.

Q. Même ReplicaSet ou nouveau ?

R. Le même : `replicas` n'est pas dans le template, donc le hash ne change pas.

Q. Après `kubectl scale --replicas=2`, le fichier dit 5. Un collègue relance `apply` : que se passe-t-il ?

R. Le cluster revient à 5 Pods ; la modification est perdue sans prévenir. C'est la dérive (drift).

Q. Quelle est la meilleure pratique ?

R. Le déclaratif : modifier le fichier, puis `apply`. `kubectl scale` reste utile en urgence, à condition de reporter ensuite le changement dans le fichier.

7. Rolling updates et rollbacks

A. Résumé du cours

Rolling update. Modifier le template (par exemple l'image) crée un *nouveau* ReplicaSet, c'est-à-dire une nouvelle *révision*. Il monte petit à petit pendant que l'ancien descend : il n'y a pas de coupure.

— **maxSurge 25 %** (arrondi au-dessus) : sur 5 Pods, 2 Pods *en plus* sont autorisés, donc jusqu'à 7.
— **maxUnavailable 25 %** (arrondi au-dessous) : au plus 1 Pod indisponible, donc au moins 4 disponibles.

Rollback. `rollout undo` *réutilise l'ancien ReplicaSet*, avec des Pods neufs. C'est un nouveau déploiement : la révision 1 devient la révision 3. Une révision = un ReplicaSet ; c'est pour cela que les anciens ReplicaSets sont gardés à 0.

Dérive après rollback. `undo` ne modifie pas le fichier. Il faut remettre le fichier d'accord avec le cluster, sinon le prochain `apply` réinstalle la mauvaise version. `apply` garde une mémoire du dernier fichier appliqué (annotation `kubectl.kubernetes.io/last-applied-configuration`), que `undo` ne met pas à jour.

B. Commandes importantes

```
sed -i 's/image: nginx$/image: nginx:1.27/' 04-deployments/deployment-nginx.yaml
kubectl apply -f 04-deployments/deployment-nginx.yaml   # lance la mise à jour
kubectl rollout status deployment monsite     # suit jusqu'à "successfully rolled out"
kubectl rollout history deployment monsite    # liste des révisions
kubectl rollout undo deployment monsite       # revient à la révision précédente
sed -i 's/image: nginx:1.27/image: nginx/' 04-deployments/deployment-nginx.yaml
```

C. Exercices pratiques

Exercice 6 : rolling update vers `nginx:1.27` [tp]

Résultats. Deux ReplicaSets : `676bf75868` (nouveau, `nginx:1.27`) avec DESIRED 5, et `86f69f946d` (ancien) avec DESIRED 0. Cinq nouveaux Pods `monsite-676bf75868-xxxxx`. Dans les Events, l'ancien descend 5, 4, 3, 2, 1, 0 pendant que le nouveau monte 0, 2, 3, 4, 5. L'annotation `revision: 2` est posée.
Appris. La mise à jour se fait sans coupure, mais deux versions coexistent pendant quelques secondes.

Exercice 7 : rollback, puis correction de la dérive [tp]

— `rollout history` : révisions 1 (`nginx`) et 2 (`nginx:1.27`).
— `rollout undo` : kubectl affiche un avertissement (la configuration d'`apply` n'est pas mise à jour). `86f69f946d` remonte à 5, avec 5 Pods neufs en `nginx`.
— `history` : révisions 2 et 3 ; la 1 est devenue la 3.
— Fichier corrigé avec `sed`, puis `apply` : `configured` (et non `unchanged`, comme je l'avais prédit). Pas de révision 4, car le template du cluster était déjà identique.

D. Questions et réponses [tp]

Q. On modifie le template : combien de ReplicaSets après `apply` ? Les Pods gardent-ils leur nom ? Y a-t-il une coupure ?

R. Deux : le nouveau à 5, l'ancien à 0. Les Pods changent de nom. Pas de coupure.

Q. Dans les Events, l'ancien ReplicaSet passe-t-il à 0 d'un coup ?

R. Non, petit à petit : cela prouve qu'il n'y a pas de coupure.

Q. Au premier événement, le nouveau ReplicaSet passe à 2 alors que l'ancien en a 5 : combien de Pods tournent ? Que veut dire « 25 % max surge » ?

R. 7 Pods. On a le droit de créer 2 Pods en plus du nombre voulu.

Q. Que garantit « 25 % max unavailable » (1 Pod sur 5) ?

R. Qu'au moins 4 Pods restent disponibles pendant la mise à jour.

Q. Pourquoi le Deployment garde-t-il l'ancien ReplicaSet à 0 ?

R. Pour pouvoir faire un rollback.

Q. Après `undo`, quel ReplicaSet passe à 5 ?

R. L'ancien, `86f69f946d`. Il n'y a pas de troisième ReplicaSet.

Q. Le fichier dit toujours `nginx:1.27` après le rollback : quel problème ?

R. Une dérive entre le fichier et le cluster.

Q. La révision 3 correspond à quelle image ? Pourquoi 3 et pas 1 ?

R. `nginx`. Un rollback est un nouveau déploiement : il reçoit un nouveau numéro.

Q. Pourquoi la révision 1 a-t-elle disparu ?

R. Elle est devenue la 3 : le même ReplicaSet a été réutilisé et renuméroté.

Q. Si quelqu'un relance `apply` demain sans corriger le fichier ?

R. Retour à `nginx:1.27` : les clients retrouvent le problème.

Q. Quelle commande supprime la dérive ?

R. `sed` pour remettre `image: nginx` dans le fichier, puis `apply`.

Q. Après la correction, `apply` affiche `created`, `configured` ou `unchanged` ?

R. Ma prédiction était `unchanged` ; le résultat réel est `configured`. `apply` compare aussi le fichier à sa mémoire (`last-applied-configuration`), restée en `nginx:1.27` après `undo`. Les Pods, eux, n'ont pas changé.

Q. Après la correction, y a-t-il une révision 4 ?

R. Non, toujours 2 et 3 : le template du cluster n'a pas changé, donc pas de nouveau ReplicaSet.

E. Erreurs et dépannage

— **Prédiction `unchanged` fausse** : c'était `configured`, à cause de l'annotation `last-applied-configuration`.
— **Avertissement après `undo`** : c'est normal, il signale la future dérive. Il faut corriger le fichier.

8. Les Services : ClusterIP, NodePort et port-forward

A. Résumé du cours

Les Pods changent d'IP quand ils sont remplacés, et ils sont plusieurs. Un **Service** donne un *nom DNS* et une *IP fixe* devant un groupe de Pods choisis par étiquette, et il répartit les connexions entre eux. C'est l'accueil du restaurant.

— **ClusterIP** (par défaut) : accessible uniquement depuis l'intérieur du cluster.
— **NodePort** : un ClusterIP *plus* un port ouvert sur le nœud (entre 30000 et 32767), accessible de l'extérieur.
— **port-forward** : un tunnel temporaire PC → cluster, relayé par l'API server. Même vers un Service, il se branche sur *un seul Pod* choisi au démarrage. C'est un outil de débogage.

Les trois ports : `nodePort` (port du nœud, 30080) → `port` (port du Service, 80) → `targetPort` (port du conteneur, 80).
L'IP d'un Service (`10.109.x.x`) n'est pas dans la plage des Pods (`10.244.x.x`) : c'est une adresse virtuelle.

B. Commandes importantes

```
kubectl apply --dry-run=client -f 06-services/service-clusterip.yaml  # test à blanc
kubectl get services                      # TYPE, CLUSTER-IP, EXTERNAL-IP, PORT(S)
kubectl describe service monsite-svc      # Selector, IP, Port, TargetPort, Endpoints
kubectl run test --image=busybox --restart=Never -- wget -qO- http://monsite-svc
kubectl logs test                         # page reçue par le client de test
kubectl delete pod test                   # ménage
minikube ip                               # IP du œnud (192.168.49.2)
curl -s http://192.168.49.2:30080         # accès NodePort depuis le PC
kubectl port-forward service/monsite-svc 8080:80 > ~/pf.log 2>&1 &   # tunnel
curl -s http://localhost:8080 | grep title   # test du tunnel
pkill -f port-forward                     # ferme le tunnel
```

C. Exercices pratiques

Exercice 8 : Service ClusterIP `monsite-svc` [tp]

Fichier `06-services/service-clusterip.yaml` (écrit à partir d'un squelette).

```
apiVersion: v1
kind: Service
metadata:
  name: monsite-svc
spec:
  type: ClusterIP
  selector:
    app: monsite          # directement sous selector, sans matchLabels
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

Résultats. `created` ; ClusterIP `10.109.84.17`, `80/TCP`, `EXTERNAL-IP <none>`. Endpoints : les 5 IP des Pods (`.19` à `.23`). Test interne avec busybox : le Pod `test` passe en `Completed` (`READY 0/1`, la commande est finie) et ses logs contiennent la page « Welcome to nginx ! ».
Appris. Le client n'a jamais connu l'IP d'un Pod : il a utilisé le nom `monsite-svc`, traduit par CoreDNS (`monsite-svc.default.svc.cluster.local`).

Exercice 9 : Service NodePort `monsite-nodeport` [tp]

Copie du ClusterIP, puis trois changements faits avec `sed` : le nom, le type (`NodePort`) et l'ajout de `nodePort: 30080`.

```
cp 06-services/service-clusterip.yaml 06-services/service-nodeport.yaml
sed -i 's/name: monsite-svc/name: monsite-nodeport/' 06-services/service-nodeport.yaml
sed -i 's/type: ClusterIP/type: NodePort/' 06-services/service-nodeport.yaml
sed -i 's/^      targetPort: 80$/&\n      nodePort: 30080/' 06-services/service-nodeport
    ↪ .yaml
```

Résultats. `NodePort 10.111.148.216 80:30080/TCP`. `curl` sur `192.168.49.2:30080` depuis le PC renvoie la page nginx.
Appris. Sans changer le nom, `apply` aurait répondu `monsite-svc unchanged` : un objet est identifié par son type et son nom.

Exercice 10 : port-forward et panne d'un Pod [tp]

Déroulé. Tunnel lancé en arrière-plan ; `curl localhost:8080` fonctionne. Puis suppression du Pod `mf7nv` (IP `.19`).
Résultats.

— Remplaçant `7t7wk`, nouvelle IP `.25`. Dans l'EndpointSlice, `.19` est retirée et `.25` ajoutée, automatiquement.
— Le NodePort continue de répondre, sans aucun changement côté client.
— Le port-forward meurt (`error: lost connection to pod`) : il était branché justement sur `mf7nv`. Il a fallu le relancer.

Appris. Le Service suit les Pods par étiquette ; le port-forward ne fait pas de répartition.

D. Questions et réponses [tp]

Q. Combien d'adresses sur la ligne Endpoints, et à quoi correspondent-elles ?

R. 5 : ce sont les 5 Pods de `monsite` (IP :port), trouvés grâce à `app=monsite`. La liste se met à jour toute seule.

Q. À quoi sert le Service `kubernetes` ?

R. C'est l'accès à l'API server (HTTPS, 443), créé automatiquement.

Q. Pourquoi `EXTERNAL-IP` vaut `<none>` ?

R. Parce que ClusterIP est interne, pas accessible de l'extérieur.

Q. Peux-tu savoir quel Pod a répondu au client busybox ?

R. Non : les 5 Pods servent la même page, et le Service choisit un Pod à chaque connexion.

Q. Qui a traduit `monsite-svc` en IP ?

R. Le DNS du cluster, CoreDNS.

Q. Le même `wget http://monsite-svc` depuis le PC marcherait-il ?

R. Non : ClusterIP est interne, et ce nom n'existe que dans le DNS du cluster.

Q. Dans `80:30080/TCP`, que représentent 80 et 30080 ?

R. 80 est le port du Service ; 30080 est le port ouvert sur le nœud.

Q. `monsite-nodeport` a-t-il aussi une CLUSTER-IP ? Qu'en conclure ?

R. Oui : NodePort inclut ClusterIP et ajoute une porte sur le nœud.

Q. Pourquoi `curl 192.168.49.2:30080` marche depuis le PC, et pas `wget http://monsite-svc` ?

R. On utilise l'IP du nœud, connue du PC, au lieu d'un nom interne ; et NodePort est accessible de l'extérieur.

Q. Pourquoi le tunnel vers un ClusterIP interne marche-t-il depuis le PC ?

R. kubectl relaie le trafic via l'API server.

Q. Le Service a-t-il suivi la panne ? Et le port-forward ?

R. Le Service, oui (Endpoints mis à jour). Le port-forward, non : il était fixé sur le Pod supprimé.

Q. Le client du NodePort a-t-il dû changer quelque chose pendant la panne ?

R. Non, rien : même IP, même port.

E. Erreurs et dépannage

Symptôme	Cause	Correction
`cat: …No such file or directory`	nano quitté sans enregistrer (seul le verrou `.swp` existait)	Ctrl+O, Entrée (« Wrote X lines »), Ctrl+X
Fichier NodePort identique au ClusterIP	la commande `cp` ou l'édition n'avait pas été lancée	`cp` avec `!`, puis `sed`
`curl -s` n'affiche rien	aucun tunnel en marche (`-s` masque l'erreur)	lancer le port-forward avec `&`, vérifier `~/pf.log`
Tunnel arrêté après la suppression d'un Pod	port-forward lié à un seul Pod	relancer le port-forward

9. Labels, selectors, Endpoints et EndpointSlices

A. Résumé du cours

Les labels sont des étiquettes clé/valeur posées sur les objets (`app: monsite`). **Les selectors** sont des filtres sur ces étiquettes. Tout le lien entre objets passe par eux, jamais par les noms ni par les IP.

Objet	Comment il choisit ses Pods
Deployment / ReplicaSet	`selector.matchLabels`, qui doit correspondre aux labels du template
ReplicaSet d'un Deployment	`app=monsite` plus `pod-template-hash` (sépare les versions)
Service	`selector` directement (sans `matchLabels`)
kubectl	`-l app=web` pour filtrer l'affichage

Endpoints / EndpointSlice : la liste à jour des IP :port des Pods *prêts* qui correspondent au selector d'un Service. Kubernetes la crée et l'entretient lui-même (par exemple `monsite-svc-7nddl`). Dans les versions récentes, l'EndpointSlice remplace l'ancien objet Endpoints.

Deux cas de diagnostic réalisés à l'oral :

— Selector `app: monsite`, Pods étiquetés `app: mon-site` : les Endpoints sont *vides* et le site est inaccessible.
— `targetPort: 8080` alors que nginx écoute sur 80 : les Endpoints sont *remplis*, mais le site ne répond pas. Des Endpoints remplis ne garantissent pas un Service qui fonctionne.

B. Commandes importantes

```
kubectl get pods -l app=monsite                    # Pods portant l'étiquette
kubectl get endpointslices -l kubernetes.io/service-name=monsite-svc
kubectl describe endpointslice -l kubernetes.io/service-name=monsite-svc | grep
    ↪ Addresses
```

[COMPLÉMENT] `kubectl get pods --show-labels` affiche toutes les étiquettes ; `kubectl label pod <nom> cle=valeur` en ajoute une.

D. Questions et réponses [tp]

Q. Que verras-tu si les Pods ont `app: mon-site` et le Service `selector: app: monsite` ?

R. Des Endpoints vides : les labels ne correspondent pas au selector. Le client ne peut pas accéder au site.

Q. `targetPort: 8080` alors que nginx écoute sur 80 : Endpoints vides ou remplis ? Le site marche-t-il ?

R. Les Endpoints sont remplis, car le selector correspond, mais le site ne marche pas : le trafic part vers un port où personne n'écoute.

10. Autres notions étudiées

10.1 Déclaratif, impératif et `apply`

— `apply` répond `created` (nouvel objet), `configured` (objet modifié) ou `unchanged` (rien à faire).
— `--dry-run=client` vérifie le fichier localement sans rien créer. `--dry-run=server` le fait valider par l'API server, sans rien enregistrer (utilisé pour valider les manifestes du dépôt).
— Les commandes impératives (`scale`, `rollout undo`, `kubectl run`) sont utiles en urgence ou pour un test, mais il faut ensuite reporter le changement dans le fichier.

10.2 Namespaces et DNS

— Mes objets sont dans le namespace `default` ; les composants système (CoreDNS…) sont dans `kube-system`.
— Nom DNS complet d'un Service : `<service>.<namespace>.svc.cluster.local`.

[COMPLÉMENT] Les namespaces seront travaillés au Jour 2.

10.3 Pod client jetable

`kubectl run test --image=busybox --restart=Never -- <commande>` crée un Pod seul qui exécute la commande puis s'arrête (`Completed`). C'est pratique pour tester un Service de l'intérieur.

10.4 Outils du terminal utilisés

```
sed -i 's/ancien/nouveau/' fichier   # remplace un texte dans un fichier
sed -i 's/^ligne$/&\nnouvelle/' f    # & = ligne trouvée ; \n = ajoute une ligne
grep mot fichier                     # lignes contenant "mot"
cmd | grep mot                       # filtre la sortie d'une commande
cp source destination                # copie un fichier
echo "texte" >> fichier              # ajoute une ligne (> écraserait le fichier)
cmd1 && cmd2                         # cmd2 seulement si cmd1 a réussi
cmd > f.log 2>&1 &                   # arrière-plan, sorties dans un fichier
pkill -f motif                       # arrête le processus dont la commande contient "
    ↪ motif"
```

10.5 Git et publication sécurisée

— Dépôt public : `K8s-learning-labs`. Commits en français, jamais de `--force`.
— `.gitignore` : kubeconfig, `*.key`, `*.crt`, `*.pem`, `.env`, dossiers minikube, `*.tar`, `*.tgz`, `*.log`.
— Les fichiers personnels sont exclus seulement en local, via `.git/info/exclude`.
— Avant chaque commit : relire la liste des fichiers et chercher les secrets (`token`, `password`, `client-key-data`, `BEGIN PRIVATE KEY`).
— Un manifeste de type Secret ne contiendra que des valeurs d'exemple (`changeme`).

```
git status                   # ce qui a changé
git add -A && git status     # tout préparer, puis vérifier la liste
git commit -m "message"      # enregistrer une version
git push                     # envoyer sur GitHub
git log --oneline            # historique des commits
```

10.6 Erreurs de méthode rencontrées

— `nano` ne marche pas avec `!` (« Standard input is not a terminal ») : utiliser `sed` ou un deuxième terminal.
— Commande envoyée sans `!` : elle n'est pas exécutée.
— `kubectl apply` sur un fichier inexistant (`the path …does not exist`) : écrire le fichier d'abord.
— Répondre par prédiction sans exécuter : toujours prouver avec la commande et la sortie.

11. Questions Kubernetes pour un entretien DevOps M2

Réponses courtes, à dire à l'oral. [TP] : question réellement posée pendant les TP ; [COMPLÉMENT] : question nouvelle.

11.1 Pods

Q. [TP] Kubernetes gère-t-il des conteneurs ou des Pods ?

R. Des Pods. Un Pod enveloppe un ou plusieurs conteneurs qui partagent le réseau et le stockage.

Q. [COMPLÉMENT] Quelle différence entre un Pod `Running` et un Pod `Ready` ?

R. `Running` : le conteneur est démarré. `Ready` : le Pod est prêt à recevoir du trafic (readiness probe réussie, colonne `READY 1/1`). Seuls les Pods Ready sont dans les Endpoints d'un Service.

Q. [COMPLÉMENT] Que signifie un `RESTARTS` qui augmente sans arrêt ?

R. Le conteneur plante en boucle, d'où l'état `CrashLoopBackOff`. On regarde `kubectl logs --previous` et les Events de `describe`.

Q. [COMPLÉMENT] Pourquoi ne crée-t-on presque jamais de Pod seul en production ?

R. Parce qu'il n'est pas relancé s'il meurt : on passe par un Deployment.

11.2 Deployments

Q. [TP] Quelle différence entre Pod, ReplicaSet et Deployment ?

R. Le Pod exécute l'application ; le ReplicaSet maintient N Pods repérés par leurs étiquettes ; le Deployment pilote les ReplicaSets (un par version) et apporte le rolling update et le rollback.

Q. [TP] Que se passe-t-il lorsqu'on supprime un Pod géré par un Deployment ?

R. Le ReplicaSet voit qu'il en manque un et en recrée un, avec un nouveau nom et une nouvelle IP. Vérifié : `f56w5` remplacé par `4vcxz`, puis `mf7nv` par `7t7wk`.

Q. [TP] Comment fonctionne un rolling update ?

R. Un nouveau ReplicaSet monte pendant que l'ancien descend, dans les limites de maxSurge et maxUnavailable : pas de coupure, mais deux versions coexistent un moment.

Q. [TP] Comment faire un rollback ? Crée-t-il une révision ?

R. `kubectl rollout undo` : il réutilise l'ancien ReplicaSet et crée une nouvelle révision (la 1 devient la 3). Ensuite, il faut corriger le fichier.

Q. [TP] Déclaratif ou impératif ?

R. Déclaratif : le fichier dans Git fait foi. L'impératif crée une dérive.

Q. [COMPLÉMENT] Comment revenir à une révision précise ?

R. `kubectl rollout undo deployment <nom> --to-revision=N`.

11.3 ReplicaSets

Q. [TP] Comment Kubernetes maintient-il 3 replicas ?

R. Le Deployment crée un ReplicaSet, qui compte en permanence les Pods portant son étiquette et compare au nombre voulu : il crée s'il en manque, supprime s'il y en a trop. C'est la boucle de réconciliation. Je l'ai testé en supprimant un Pod.

Q. [TP] Pourquoi garder les anciens ReplicaSets à 0 ?

R. Pour le rollback : une révision = un ReplicaSet.

Q. [TP] Un ReplicaSet peut-il prendre un Pod qu'il n'a pas créé ?

R. Oui, il l'adopte s'il porte la bonne étiquette (`Controlled By`). Il peut aussi supprimer un Pod en trop.

11.4 Services

Q. [TP] Pourquoi utiliser un Service plutôt que l'IP des Pods ?

R. Les IP des Pods changent. Le Service offre un nom DNS et une IP stables, et répartit le trafic entre les Pods prêts.

Q. [TP] Quelle différence entre ClusterIP, NodePort et port-forward ?

R. ClusterIP : interne. NodePort : ClusterIP plus un port sur chaque nœud (30000–32767), pour un accès externe. port-forward : tunnel temporaire via l'API server vers un seul Pod, pour déboguer.

Q. [TP] Comment un Service sélectionne-t-il ses Pods ?

R. Par son `selector`, comparé aux labels des Pods ; le résultat est la liste des Endpoints (EndpointSlice), mise à jour automatiquement.

Q. [COMPLÉMENT] Et en production, comment expose-t-on une application ?

R. Avec un Service `LoadBalancer` (load balancer du cloud) ou un `Ingress` (routage HTTP par nom de domaine ou chemin), plutôt qu'avec NodePort.

Q. [COMPLÉMENT] Que sont `port`, `targetPort` et `nodePort` ?

R. `port` : port du Service ; `targetPort` : port du conteneur ; `nodePort` : port ouvert sur le nœud.

11.5 Réseau et dépannage

Q. [TP] Que vérifier lorsqu'un Service est inaccessible alors que les Pods sont Running ?

R. Dans l'ordre : 1) les Endpoints (vides : selector différent des labels) ; 2) `targetPort` = port du conteneur ; 3) les Pods sont-ils Ready ; 4) test depuis l'intérieur avec busybox, pour séparer un problème de Service d'un problème d'accès externe ; 5) `logs` et `describe` des Pods.

Q. [TP] Que se passe-t-il si les labels ne correspondent pas au selector ?

R. Les Endpoints sont vides : le Service n'envoie le trafic nulle part.

Q. [TP] Que se passe-t-il si `targetPort` ne correspond pas au port de nginx ?

R. Les Endpoints sont remplis, mais les connexions échouent : personne n'écoute sur ce port.

Q. [TP] Qui traduit le nom d'un Service en IP ?

R. CoreDNS, le DNS du cluster.

Q. [COMPLÉMENT] Un Pod reste en `Pending` : que regarder ?

R. Les Events de `kubectl describe pod` : pas de nœud disponible, ressources insuffisantes, volume manquant…

Q. [COMPLÉMENT] Un Pod est en `ImagePullBackOff` : que faire ?

R. Vérifier le nom et le tag de l'image, l'accès au registre et les identifiants, dans les Events de `describe`.

12. Fiche récapitulative des commandes

TP : déjà utilisée ; Compl. : à connaître pour l'entretien (pas encore pratiquée).

Commande	Fonction	État
Cluster et minikube		
`minikube start --driver=docker`	crée ou rallume le cluster	TP
`minikube status`	état du cluster	TP
`minikube stop`	éteint le cluster	TP
`minikube ip`	IP du nœud	TP
`kubectl get nodes`	nœuds et état Ready	TP
`kubectl version --client`	version de kubectl	TP
`kubectl cluster-info`	adresse de l'API server	Compl.
`kubectl config get-contexts`	clusters configurés	Compl.
`minikube service <svc> --url`	URL d'accès à un NodePort	Compl.
Pods		
`kubectl get pods`	liste des Pods	TP
`kubectl get pods -o wide`	avec IP et nœud	TP
`kubectl get pods -l app=web`	filtre par étiquette	TP
`kubectl describe pod <nom>`	fiche et Events	TP
`kubectl delete pod <nom>`	supprime un Pod	TP
`kubectl run test --image=busybox --restart=Never -- <cmd>`	Pod jetable de test	TP
`kubectl get pods -w`	suit les changements en direct	Compl.
`kubectl get pods --show-labels`	affiche les étiquettes	Compl.
`kubectl exec -it <pod> -- sh`	shell dans le conteneur	Compl.
Deployments		
`kubectl get deployments`	état READY / UP-TO-DATE	TP
`kubectl describe deployment <nom>`	stratégie et Events	TP
`kubectl get deployments,replicasets,pods`	les trois niveaux	TP
`kubectl create deployment <nom> --image=nginx --dry-run=client -o yaml`	génère un YAML de départ	Compl.
ReplicaSets		

Commande	Fonction	État
`kubectl get replicasets`	DESIRED / CURRENT / READY	TP
`kubectl describe replicaset <nom>`	Controlled By, Selector	TP
`kubectl delete replicaset <nom>`	supprime le ReplicaSet et ses Pods	TP
Services et réseau		
`kubectl get services`	liste des Services	TP
`kubectl describe service <nom>`	Selector, ports, Endpoints	TP
`kubectl get endpointslices -l kubernetes.io/service-name=<svc>`	Pods derrière le Service	TP
`kubectl port-forward service/<svc> 8080:80`	tunnel PC vers un Pod	TP
`kubectl port-forward pod/<pod> 8080:80`	tunnel vers un Pod précis	Compl.
`curl -s http://<ip-noeud>:30080`	teste un NodePort	TP
`kubectl expose deployment <nom> --port=80 --type=NodePort`	crée un Service en impératif	Compl.
Scaling et mises à jour		
`kubectl apply -f <fichier>`	applique l'état souhaité	TP
`kubectl scale deployment <nom> --replicas=N`	scaling impératif	TP*
`kubectl rollout status deployment <nom>`	suit une mise à jour	TP
`kubectl rollout history deployment <nom>`	liste des révisions	TP
`kubectl rollout undo deployment <nom>`	rollback	TP
`kubectl rollout undo deployment <nom> --to-revision=N`	rollback vers une révision	Compl.
`kubectl set image deployment/<nom> nginx=nginx:1.27`	change l'image (impératif)	Compl.
`kubectl rollout restart deployment <nom>`	recrée tous les Pods	Compl.
Logs et dépannage		
`kubectl logs <pod>`	sortie du conteneur	TP
`kubectl logs -f <pod>`	suit les logs en direct	Compl.
`kubectl logs --previous <pod>`	logs du conteneur précédent (crash)	Compl.
`kubectl get events --sort-by=.metadata.creationTimestamp`	événements récents	Compl.
`kubectl get pods -o custom-columns= NOM:.metadata.name,IMAGE:.spec. containers[0].image`	colonnes choisies	TP
`kubectl explain deployment.spec`	documentation intégrée	Compl.
Fichiers YAML et nettoyage		
`kubectl apply --dry-run=client -f <fichier>`	vérifie sans créer	TP
`kubectl apply --dry-run=server -f <fichier>`	validation par l'API server	TP
`kubectl delete -f <fichier>`	supprime ce que décrit le fichier	TP
`kubectl get <objet> <nom> -o yaml`	affiche l'objet complet	Compl.
`kubectl diff -f <fichier>`	différences fichier / cluster	Compl.
`kubectl get all`	principaux objets du namespace	Compl.

* Commande étudiée ; son exécution n'est pas attestée dans les notes.